

Sequential Task:

Social Media

Object-Oriented Programming and GUI

April 23, 2026

1. Implement the code fragment marked with the *TODO 01* label, responsible for creating a data stream and printing its elements to the console.
 - (a) Create a stream of elements of type `String` using the `Stream.of(...)` method.
 - Pass three text values representing user identifiers as arguments: `"anna_k"`, `"bartek99"`, `"zosia_m"`.
 - (b) Invoke the terminal operation `forEach(...)` on the created stream.
 - (c) Pass the method reference `System.out::println` to the `forEach(...)` method.
 - Ensure that each stream element is printed on a separate line in the console.
2. Implement the code fragment marked with the *TODO 02* label, responsible for processing the `tags` array using streams and printing the transformed values.
 - (a) Create a stream based on the `tags` array using the `Arrays.stream(...)` method.
 - (b) Invoke the intermediate operation `map(...)` to transform each element of the stream.
 - Use the method reference `String::toUpperCase`.
 - Ensure that each text element is converted to uppercase.
 - (c) Invoke the terminal operation `forEach(...)` on the processed stream.
 - (d) Pass the method reference `System.out::println`.
 - Ensure that each transformed element is printed on a separate line in the console.
3. Implement the code fragment marked with the *TODO 03* label, generating a sequence of values, transforming them, and printing the result on a single line.
 - (a) Create an infinite stream of integers using the `Stream.iterate(...)` method.
 - Set the initial value to `1`.
 - Define the function generating subsequent elements as `n -> n + 1` in order to obtain an increasing sequence of numbers.
 - (b) Limit the stream to the first ten elements using the `limit(10)` operation.
 - (c) Transform each element of the stream into a text form using the `map(...)` operation.
 - Define the lambda expression `n -> "POST_" + n`.

- Create textual identifiers based on consecutive numbers.
- (d) Invoke the terminal operation `forEach(...)`.
- (e) Inside the lambda expression, use `System.out.print(...)` instead of `println`.
- Print each element on the same line.
 - Add a space after each element to separate the values.
4. Implement the code fragment marked with the *TODO 04* label, generating random values and printing them on a single line.
- (a) Create an infinite stream of integers using the `Stream.generate(...)` method.
- Pass the lambda expression `() -> random.nextInt(101)` as the generator.
 - Ensure that random numbers from 0 to 100, inclusive, are generated.
- (b) Limit the stream to five elements using the `limit(5)` operation.
- (c) Invoke the terminal operation `forEach(...)`.
- (d) Inside the lambda expression, use `System.out.print(...)`.
- Print each generated number on the same line.
 - Add a space after each number to separate the values.
5. Implement the code fragment marked with the *TODO 05* label, demonstrating the use of an empty stream and an operation returning a result in the form of `Optional`.
- (a) Create an empty stream of type `String` using the `Stream.empty()` method and assign it to the `emptyStream` variable.
- (b) Invoke the terminal operation `findFirst()` on the `emptyStream`.
- (c) Store the result of the `findFirst()` method in the variable `result` of type `Optional<String>`.
- (d) Take into account that for an empty stream, the result will be an empty `Optional` object.
- (e) Uncomment the lines printing information to the console.
6. Implement the code fragment marked with the *TODO 06* label, demonstrating the creation of a collection and the processing of its elements using a stream.
- (a) Create an immutable list of type `String` using the `List.of(...)` method and assign it to the variable `usernames`.
- Pass several example usernames as the list elements.
- (b) Invoke the `stream()` method on the `usernames` object to obtain a stream of type `Stream<String>`.
- (c) Invoke the terminal operation `forEach(...)` on the created stream.
- (d) Pass the method reference `System.out::println`.
- Ensure that each list element is printed on a separate line in the console.
7. Check whether the program output will be:

```

=====
Problem P1 - Creating Streams
=====

--- [1] Stream.of() ---
anna_k
bartek99
zosia_m

--- [2] Arrays.stream() ---
JAVA
NEWS
PHOTO
TECH

--- [3] Stream.iterate() - post IDs ---
POST_1 POST_2 POST_3 POST_4 POST_5 POST_6 POST_7 POST_8 POST_9 POST_10

--- [4] Stream.generate() - random likes ---
15 59 4 89 11

--- [5] Stream.empty() + Optional ---
Does result exist: false
Value or default: No posts

--- [BONUS] Collection.stream() ---
anna_k
bartek99
zosia_m

```

8.  Milestone 1: **present the results of your work to the instructor**
9. Implement the code fragment marked with the *TODO 07* label, processing the **posts** collection using streams, filtering the data, and transforming its content before displaying it.
 - (a) Invoke the **stream()** method on the **posts** collection to obtain a stream of elements.
 - (b) Apply the intermediate operation **filter(...)** in order to limit the stream to selected elements.
 - Define the lambda expression **p -> p.getCategory().equals("TECH")**.
 - Ensure that only objects whose category is "TECH" remain in the stream.
 - (c) Apply the intermediate operation **map(...)** in order to transform the content of the elements.
 - Retrieve the post content by invoking **p.getContent()**.
 - Determine the maximum fragment length as the minimum of 40 and the text length.
 - Use the **substring(0, ...)** method to retrieve the initial fragment of the text.
 - Return the shortened version of the content as the result of the mapping.
 - (d) Invoke the terminal operation **forEach(...)** on the processed stream.

- (e) Pass the method reference `System.out::println`.
 - Ensure that the shortened content of each filtered post is printed on a separate line.
10. Implement the code fragment marked with the *TODO 08* label, processing the `posts` collection in order to obtain unique usernames of post authors.
- (a) Invoke the `stream()` method on the `posts` collection to obtain a stream of objects.
 - (b) Apply the intermediate operation `map(...)` in order to transform the stream elements.
 - Define the lambda expression `p -> p.getAuthor().getUsername()`.
 - Transform each `Post` object into the author's username of type `String`.
 - (c) Apply the intermediate operation `distinct()` to remove duplicate usernames.
 - (d) Invoke the terminal operation `forEach(...)`.
 - (e) Pass the method reference `System.out::println`.
 - Print each unique username on a separate line.
11. Implement the code fragment marked with the *TODO 09* label, processing the `posts` collection, filtering posts by the number of likes, and formatting the result as text.
- (a) Invoke the `stream()` method on the `posts` collection to obtain a stream of objects.
 - (b) Apply the intermediate operation `filter(...)` in order to limit the stream to popular posts.
 - Define the lambda expression `p -> p.getLikes() > 200`.
 - Keep only those posts whose number of likes exceeds 200.
 - (c) Apply the intermediate operation `map(...)` in order to transform the objects into text form.
 - Use `String.format("%-12s -> %d", ...)`.
 - Pass the author's username through `p.getAuthor().getUsername()`.
 - Pass the number of likes through `p.getLikes()`.
 - Ensure that the username is aligned to a width of 12 characters.
 - (d) Invoke the terminal operation `forEach(...)`.
 - (e) Pass the method reference `System.out::println`.
 - Print each formatted line on a separate line in the console.
12. Implement the code fragment marked with the *TODO 10* label, processing the `posts` collection, transforming the post content, and saving the result into a new list.
- (a) Invoke the `stream()` method on the `posts` collection to obtain a stream of objects of type `Post`.
 - (b) Apply the intermediate operation `map(...)` in order to transform `Post` objects into their textual content.

- Use the method reference `Post::getContent`.
 - Obtain a stream of values of type `String`.
- (c) Apply another `map(...)` operation to transform the text to uppercase.
- Use the method reference `String::toUpperCase`.
 - Obtain a stream of transformed strings.
- (d) Apply the terminal operation `collect(...)` in order to gather the results into a collection.
- Use `Collectors.toList()`.
 - Assign the result to the variable `upperContents` of type `List<String>`.
- (e) Invoke the `forEach(...)` method on the `upperContents` list.
- (f) Pass the method reference `System.out::println`.
- Print each transformed content string on a separate line.

13. Check whether the program output will be:

```
=====
Problem P1 - Creating Streams
=====
--- [1] Stream.of() ---
anna_k
bartek99
zosia_m

--- [2] Arrays.stream() ---
JAVA
NEWS
PHOTO
TECH

--- [3] Stream.iterate() - post IDs ---
POST_1 POST_2 POST_3 POST_4 POST_5 POST_6 POST_7 POST_8 POST_9 POST_10

--- [4] Stream.generate() - random likes ---
15 59 4 89 11

--- [5] Stream.empty() + Optional ---
Does result exist: false
Value or default: No posts

--- [BONUS] Collection.stream() ---
anna_k
bartek99
zosia_m

=====
Problem P2 - filter(), map(), distinct()
=====
--- [1] Posts in TECH category (first 40 characters) ---
```

```
Java Stream API to świetna sprawa!  
Mój ulubiony #java trick - metoda reduce  
Docker + Spring Boot w 10 minut  
Czy AI zastąpi programistów?  
Wydażność streamów równoległych w Javie
```

```
--- [2] Unique post authors (distinct) ---
```

```
anna_k  
bartek99  
zosia_m  
tomek_dev  
kasia.foto  
pawel_x
```

```
--- [3] Posts with more than 200 likes ---
```

```
anna_k      -> 245  
zosia_m     -> 523  
anna_k      -> 310  
kasia.foto  -> 892  
zosia_m     -> 445  
bartek99    -> 201
```

```
--- [4] Post contents in UPPERCASE ---
```

```
JAVA STREAM API TO ŚWIETNA SPRAWA!  
KRAKÓW DZIŚ PIĘKNY :)  
NOWE PRZEPISY PODATKOWE - CO MUSISZ WIEDZIEĆ  
MÓJ ULUBIONY #JAVA TRICK - METODA REDUCE()  
DOCKER + SPRING BOOT W 10 MINUT  
ZŁOTA GODZINA - MOJE ZDJĘCIA Z WROCŁAWIA  
CZY AI ZASTĄPI PROGRAMISTÓW?  
BREAKING NEWS - WYBORY SAMORZĄDOWE  
TOP 5 KAWIARNI W KRAKOWIE  
WYDAJNOŚĆ STREAMÓW RÓWNOLEGŁYCH W JAVIE
```

14. 🚩 Milestone 2: **present the results of your work to the instructor**
15. Implement the code fragment marked with the *TODO 11* label, processing the `posts` collection, selecting the most popular posts, and presenting them in an ordered form.
 - (a) Invoke the `stream()` method on the `posts` collection to obtain a stream of objects of type `Post`.
 - (b) Apply the intermediate operation `sorted(...)` in order to sort the elements.
 - Use `Comparator.comparingInt(Post::getLikes)` to compare the number of likes.
 - Invoke the `reversed()` method in order to obtain descending order by the number of likes.
 - (c) Limit the stream to three elements using the `limit(3)` operation.
 - (d) Apply the intermediate operation `map(...)` in order to transform the objects into text form.

- Use `String.format("%-12s -> %d", ...)`.
 - Pass the author's username through `p.getAuthor().getUsername()`.
 - Pass the number of likes through `p.getLikes()`.
 - Ensure that the username is aligned to a width of 12 characters.
- (e) Invoke the terminal operation `forEach(...)`.
- (f) Pass the method reference `System.out::println`.
- Print the three most popular posts on separate lines.
16. Implement the code fragment marked with the *TODO 12* label, processing the `posts` collection, sorting the posts by publication date, and presenting a shortened version of each post.
- (a) Invoke the `stream()` method on the `posts` collection to obtain a stream of objects of type `Post`.
- (b) Apply the intermediate operation `sorted(...)` in order to arrange the elements.
- Use `Comparator.comparing(Post::getPublishedAt)` to compare publication dates.
 - Invoke the `reversed()` method in order to obtain order from the newest to the oldest posts.
- (c) Apply the intermediate operation `map(...)` in order to transform the objects into text form.
- Use `String.format("%s | %s", ...)`.
 - Pass the publication date through `p.getPublishedAt()`.
 - Retrieve the post content through `p.getContent()`.
 - Determine the fragment length as the minimum of 35 and the text length.
 - Use `substring(0, ...)` in order to shorten the content.
 - Pass the shortened content as the second formatting argument.
- (d) Invoke the terminal operation `forEach(...)`.
- (e) Pass the method reference `System.out::println`.
- Print each formatted post on a separate line.
17. Implement the code fragment marked with the *TODO 13* label, processing the `posts` collection, skipping the initial elements, and presenting the remaining ones in a shortened, formatted form.
- (a) Invoke the `stream()` method on the `posts` collection to obtain a stream of objects of type `Post`.
- (b) Apply the intermediate operation `skip(3)` to skip the first three elements of the stream.
- (c) Apply the intermediate operation `map(...)` in order to transform the objects into text form.

- Use `String.format("[%s] %s", ...)`.
 - Pass the post identifier through `p.getId()`.
 - Retrieve the post content through `p.getContent()`.
 - Determine the fragment length as the minimum of 35 and the text length.
 - Use the `substring(0, ...)` method to obtain a shortened content fragment.
 - Pass the shortened text as the second formatting argument.
- (d) Invoke the terminal operation `forEach(...)`.
- (e) Pass the method reference `System.out::println`.
- Print each formatted post on a separate line.
18. Implement the code fragment marked with the *TODO 14* label, performing pagination on the `posts` collection and presenting the results in a formatted form.
- (a) Invoke the `stream()` method on the `posts` collection to obtain a stream of objects of type `Post`.
- (b) Apply the intermediate operation `skip(...)` in order to skip elements belonging to previous pages.
- Compute the number of elements to skip as `(page - 1) * pageSize`.
 - Cast the result to type `long`.
- (c) Apply the intermediate operation `limit(pageSize)` to limit the stream to the number of elements corresponding to the page size.
- (d) Apply the terminal operation `collect(...)` in order to gather the results into a list.
- Use `Collectors.toList()`.
 - Assign the result to the variable `pageResult` of type `List<Post>`.
- (e) Invoke the `forEach(...)` method on the `pageResult` list.
- (f) Inside the lambda expression, use `System.out.println(...)` to print each element.
- (g) Implement text formatting using `String.format("[%s] %s (%d)", ...)`.
- Pass the post identifier through `p.getId()`.
 - Retrieve the post content through `p.getContent()` and shorten it to a maximum of 35 characters using `substring(0, Math.min(...))`.
 - Pass the number of likes through `p.getLikes()`.
19. Implement the code fragment marked with the *TODO 15* label, processing the `posts` collection, sorting posts by multiple criteria, and presenting them in formatted text form.
- (a) Invoke the `stream()` method on the `posts` collection to obtain a stream of objects of type `Post`.
- (b) Apply the intermediate operation `sorted(...)` in order to arrange the elements using a composite comparator.
- i. Create the primary sorting criterion using `Comparator.comparing(Post::getCategory)`.

- ii. Add the second sorting criterion by invoking `thenComparing(...)`.
 - iii. Inside `thenComparing(...)` use `Comparator.comparingInt(Post::getLikes).reversed()`.
 - iv. Ensure that posts are sorted first in ascending order by category and, within the same category, in descending order by the number of likes.
- (c) Apply the intermediate operation `map(...)` to transform each `Post` object into text form.
- Use `String.format("%-10s | %-12s | %d", ...)`.
 - Pass the post category through `p.getCategory()`.
 - Pass the author's username through `p.getAuthor().getUsername()`.
 - Pass the number of likes through `p.getLikes()`.
 - Preserve column alignment according to the widths specified in the format.
- (d) Invoke the terminal operation `forEach(...)` on the processed stream.
- (e) Pass the method reference `System.out::println`.
- Ensure that each formatted line is printed in a separate row of the console.

20. Check whether the program output will be:

```
=====
Problem P1 - Creating Streams
=====
--- [1] Stream.of() ---
anna_k
bartek99
zosia_m

--- [2] Arrays.stream() ---
JAVA
NEWS
PHOTO
TECH

--- [3] Stream.iterate() - post IDs ---
POST_1 POST_2 POST_3 POST_4 POST_5 POST_6 POST_7 POST_8 POST_9 POST_10

--- [4] Stream.generate() - random likes ---
15 59 4 89 11

--- [5] Stream.empty() + Optional ---
Does result exist: false
Value or default: No posts

--- [BONUS] Collection.stream() ---
anna_k
bartek99
zosia_m

=====
Problem P2 - filter(), map(), distinct()
```

```

=====
--- [1] Posts in TECH category (first 40 characters) ---
Java Stream API to świetna sprawa!
Mój ulubiony #java trick - metoda reduce
Docker + Spring Boot w 10 minut
Czy AI zastąpi programistów?
Wydajność streamów równoległych w Javie

--- [2] Unique post authors (distinct) ---
anna_k
bartek99
zosia_m
tomek_dev
kasia.foto
pawel_x

--- [3] Posts with more than 200 likes ---
anna_k      -> 245
zosia_m     -> 523
anna_k      -> 310
kasia.foto  -> 892
zosia_m     -> 445
bartek99    -> 201

--- [4] Post contents in UPPERCASE ---
JAVA STREAM API TO ŚWIETNA SPRAWA!
KRAKÓW DZIŚ PIĘKNY :)
NOWE PRZEPISY PODATKOWE - CO MUSISZ WIEDZIEĆ
MÓJ ULUBIONY #JAVA TRICK - METODA REDUCE()
DOCKER + SPRING BOOT W 10 MINUT
ZŁOTA GODZINA - MOJE ZDJĘCIA Z WROCŁAWIA
CZY AI ZASTĄPI PROGRAMISTÓW?
BREAKING NEWS - WYBORY SAMORZĄDOWE
TOP 5 KAWIARNI W KRAKOWIE
WYDAJNOŚĆ STREAMÓW RÓWNOLEGLYCH W JAVIE

=====
Problem P3 - sorted(), limit(), skip()
=====

--- [1] TOP 3 posts by likes ---
kasia.foto  -> 892
zosia_m     -> 523
zosia_m     -> 445

--- [2] Posts from newest ---
2024-03-17 | Wydajność streamów równoległych w J
2024-03-16 | Top 5 kawiarni w Krakowie
2024-03-15 | Czy AI zastąpi programistów?
2024-03-15 | Breaking news - wybory samorządowe
2024-03-14 | Docker + Spring Boot w 10 minut
2024-03-14 | Złota godzina - moje zdjęcia z Wroc
2024-03-13 | Mój ulubiony #java trick - metoda r

```

2024-03-12 | Nowe przepisy podatkowe - co musisz
2024-03-11 | Kraków dziś piękny :)
2024-03-10 | Java Stream API to świetna sprawa!

--- [3] Posts starting from the 4th (skip 3) ---

[p4] Mój ulubiony #java trick - metoda r
[p5] Docker + Spring Boot w 10 minut
[p6] Złota godzina - moje zdjęcia z Wroc
[p7] Czy AI zastąpi programistów?
[p8] Breaking news - wybory samorządowe
[p9] Top 5 kawiarni w Krakowie
[p10] Wydajność streamów równoległych w J

--- [4] Pagination - page 2, size 4 ---

[p5] Docker + Spring Boot w 10 minut (198)
[p6] Złota godzina - moje zdjęcia z Wroc (892)
[p7] Czy AI zastąpi programistów? (134)
[p8] Breaking news - wybory samorządowe (445)

--- [BONUS] Sorting: category -> likes descending ---

LIFESTYLE	kasia.foto	892
LIFESTYLE	bartek99	201
LIFESTYLE	bartek99	87
NEWS	zosia_m	523
NEWS	zosia_m	445
TECH	anna_k	310
TECH	anna_k	245
TECH	tomek_dev	198
TECH	anna_k	178
TECH	pawel_x	134

21. 🚩 Milestone 3: **present the results of your work to the instructor**
22. Implement the code fragment marked with the *TODO 16* label, calculating the number of posts belonging to a specified category.
 - (a) Invoke the `stream()` method on the `posts` collection to obtain a stream of objects of type `Post`.
 - (b) Apply the intermediate operation `filter(...)` in order to limit the stream to selected elements.
 - Define the lambda expression `p -> p.getCategory().equals("NEWS")`.
 - Leave in the stream only those posts whose category is "NEWS".
 - (c) Invoke the terminal operation `count()` to count the number of elements satisfying the condition.
 - (d) Store the result of type `long` in the variable `newsCount`.
 - (e) Uncomment the line displaying information in the console.
23. Implement the code fragment marked with the *TODO 17* label, checking whether there is at least one post in the `posts` collection that satisfies a given condition.

- (a) Invoke the `stream()` method on the `posts` collection to obtain a stream of objects of type `Post`.
 - (b) Apply the terminal operation `anyMatch(...)`.
 - (c) Define the lambda expression `p -> p.getTags().contains("docker")`.
 - Retrieve the list of tags through `p.getTags()`.
 - Check whether it contains the value `"docker"`.
 - (d) Store the result of the operation in the variable `hasDocker` of type `boolean`.
 - Expect the value `true` if there is at least one matching post.
 - Otherwise, expect the value `false`.
 - (e) Uncomment the line displaying information in the console.
24. Implement the code fragment marked with the *TODO 18* label, checking whether all elements of the `posts` collection satisfy a given condition.
- (a) Invoke the `stream()` method on the `posts` collection to obtain a stream of objects of type `Post`.
 - (b) Apply the terminal operation `allMatch(...)`.
 - (c) Define the lambda expression `p -> p.getComments() > 0`.
 - Retrieve the number of comments through `p.getComments()`.
 - Check whether the number of comments is greater than 0.
 - (d) Store the result of the operation in the variable `allHaveComments` of type `boolean`.
 - Expect the value `true` if every post has at least one comment.
 - Otherwise, expect the value `false`.
 - (e) Uncomment the line displaying information in the console.
25. Implement the code fragment marked with the *TODO 19* label, checking whether no element of the `posts` collection satisfies a negative condition.
- (a) Invoke the `stream()` method on the `posts` collection to obtain a stream of objects of type `Post`.
 - (b) Apply the terminal operation `noneMatch(...)`.
 - (c) Define the lambda expression `p -> p.getLikes() < 0`.
 - Retrieve the number of likes through `p.getLikes()`.
 - Check whether the value is less than 0.
 - (d) Store the result of the operation in the variable `noNegativeLikes` of type `boolean`.
 - Expect the value `true` if no post has a negative number of likes.
 - Otherwise, expect the value `false`.
 - (e) Uncomment the line displaying information in the console.
26. Implement the code fragment marked with the *TODO 20* label, searching for the first post of a given category and safely handling the result using the `Optional` class.

- (a) Invoke the `stream()` method on the `posts` collection to obtain a stream of objects of type `Post`.
 - (b) Apply the intermediate operation `filter(...)`.
 - Define the lambda expression `p -> p.getCategory().equals("LIFESTYLE")`.
 - Restrict the stream to posts whose category is "LIFESTYLE".
 - (c) Invoke the terminal operation `findFirst()`.
 - (d) Store the result in the variable `firstLifestyle` of type `Optional<Post>`.
27. Implement safe result handling using the `ifPresentOrElse(...)` method.
- (a) Invoke the `ifPresentOrElse(...)` method on the `firstLifestyle` object.
 - (b) Define the first argument as a lambda expression handling the case where an element is found.
 - Print a message containing the post content through `p.getContent()`.
 - (c) Define the second argument as a lambda expression handling the absence of a result.
 - Print a message informing that there are no posts in the "LIFESTYLE" category.
28. Implement an alternative way of retrieving the value using `Optional`.
- (a) Invoke the `map(...)` method on the `firstLifestyle` object.
 - Use the method reference `Post::getContent` to transform `Optional<Post>` into `Optional<String>`.
 - (b) Invoke the `orElse("Brak treści")` method.
 - Ensure that a default value is returned when no result is found.
 - (c) Assign the result to the variable `content` of type `String`.
 - (d) Uncomment the line displaying information in the console.
29. Implement the code fragment marked with the *TODO 21* label, searching for the first user satisfying a compound condition and handling the result using `Optional`.
- (a) Invoke the `stream()` method on the `users` collection to obtain a stream of objects of type `User`.
 - (b) Apply the intermediate operation `filter(...)`.
 - Define the lambda expression `u -> !u.isVerified() && u.getCity().equals("Kraków")`.
 - Restrict the stream to users who are not verified and come from the city of "Kraków".
 - (c) Invoke the terminal operation `findFirst()`.
 - (d) Store the result in the variable `unverifiedKrakow` of type `Optional<User>`.
30. Implement result handling using the `ifPresentOrElse(...)` method.
- (a) Invoke the `ifPresentOrElse(...)` method on the `unverifiedKrakow` object.

- (b) Define the first argument as a lambda expression handling the case where a user is found.
 - Print a message containing the username through `u.getUsername()`.
 - (c) Define the second argument as a lambda expression handling the absence of a result.
 - Print a message informing that no matching user was found.
31. Implement the code fragment marked with the *TODO 22* label, searching for a post satisfying a condition and presenting different ways of handling a result of type `Optional`.
- (a) Invoke the `stream()` method on the `posts` collection.
 - (b) Apply the `filter(...)` operation.
 - Define the condition `p -> p.getLikes() > 1000`.
 - Restrict the stream to posts with more than 1000 likes.
 - (c) Invoke the `findFirst()` operation.
 - (d) Store the result in the variable `maybePost` of type `Optional<Post>`.
 - (e) Take into account that the result may be empty.
32. Implement a demonstration of the methods of the `Optional` class.
- (a) Invoke `maybePost.isPresent()` and print the result.
 - Expect the value `false` if no element is found.
 - (b) Invoke `maybePost.orElse(null)` and print the result.
 - Return `null` when the value does not exist.
 - (c) Invoke `maybePost.orElseGet(...)`.
 - Pass a lambda expression creating a new `Post` object.
 - Set default values, including the content `"Brak"`.
 - Invoke `getContent()` on the result.
 - Print the obtained value.
33. Implement handling of the case where the value is missing using an exception.
- (a) Invoke the method `maybePost.orElseThrow(...)`.
 - (b) Pass a lambda expression creating a `RuntimeException` with the message `"Post nie istnieje!"`.
 - (c) Place the call inside a `try` block.
 - (d) Add a `catch(RuntimeException e)` block.
 - (e) Inside the `catch` block, print a message containing `e.getMessage()`.
34. Check whether the program output will be:

```

=====
Problem P1 - Creating Streams
=====

--- [1] Stream.of() ---
anna_k
bartek99
zosia_m

--- [2] Arrays.stream() ---
JAVA
NEWS
PHOTO
TECH

--- [3] Stream.iterate() - post IDs ---
POST_1 POST_2 POST_3 POST_4 POST_5 POST_6 POST_7 POST_8 POST_9 POST_10

--- [4] Stream.generate() - random likes ---
15 59 4 89 11

--- [5] Stream.empty() + Optional ---
Does result exist: false
Value or default: No posts

--- [BONUS] Collection.stream() ---
anna_k
bartek99
zosia_m

=====
Problem P2 - filter(), map(), distinct()
=====

--- [1] Posts in TECH category (first 40 characters) ---
Java Stream API to świetna sprawa!
Mój ulubiony #java trick - metoda reduce
Docker + Spring Boot w 10 minut
Czy AI zastąpi programistów?
Wydajność streamów równoległych w Javie

--- [2] Unique post authors (distinct) ---
anna_k
bartek99
zosia_m
tomek_dev
kasia.foto
pawel_x

--- [3] Posts with more than 200 likes ---
anna_k      -> 245
zosia_m     -> 523
anna_k      -> 310
kasia.foto  -> 892

```

zosia_m -> 445
bartek99 -> 201

--- [4] Post contents in UPPERCASE ---
JAVA STREAM API TO ŚWIETNA SPRAWA!
KRAKÓW DZIŚ PIĘKNY :)
NOWE PRZEPISY PODATKOWE - CO MUSISZ WIEDZIEĆ
MÓJ ULUBIONY #JAVA TRICK - METODA REDUCE()
DOCKER + SPRING BOOT W 10 MINUT
ZŁOTA GODZINA - MOJE ZDJĘCIA Z WROCŁAWIA
CZY AI ZASTĄPI PROGRAMISTÓW?
BREAKING NEWS - WYBORY SAMORZĄDOWE
TOP 5 KAWIARNI W KRAKOWIE
WYDAJNOŚĆ STREAMÓW RÓWNOLEGŁYCH W JAVIE

=====

```
Problem P3 - sorted(), limit(), skip()
```

=====

--- [1] TOP 3 posts by likes ---

kasia.foto -> 892
zosia_m -> 523
zosia_m -> 445

--- [2] Posts from newest ---

2024-03-17 | Wydajność streamów równoległych w J
2024-03-16 | Top 5 kawiarni w Krakowie
2024-03-15 | Czy AI zastąpi programistów?
2024-03-15 | Breaking news - wybory samorządowe
2024-03-14 | Docker + Spring Boot w 10 minut
2024-03-14 | Złota godzina - moje zdjęcia z Wroc
2024-03-13 | Mój ulubiony #java trick - metoda r
2024-03-12 | Nowe przepisy podatkowe - co musisz
2024-03-11 | Kraków dziś piękny :)
2024-03-10 | Java Stream API to świetna sprawa!

--- [3] Posts starting from the 4th (skip 3) ---

[p4] Mój ulubiony #java trick - metoda r
[p5] Docker + Spring Boot w 10 minut
[p6] Złota godzina - moje zdjęcia z Wroc
[p7] Czy AI zastąpi programistów?
[p8] Breaking news - wybory samorządowe
[p9] Top 5 kawiarni w Krakowie
[p10] Wydajność streamów równoległych w J

--- [4] Pagination - page 2, size 4 ---

[p5] Docker + Spring Boot w 10 minut (198)
[p6] Złota godzina - moje zdjęcia z Wroc (892)
[p7] Czy AI zastąpi programistów? (134)
[p8] Breaking news - wybory samorządowe (445)

--- [BONUS] Sorting: category -> likes descending ---

LIFESTYLE | kasia.foto | 892

LIFESTYLE	bartek99	201
LIFESTYLE	bartek99	87
NEWS	zosia_m	523
NEWS	zosia_m	445
TECH	anna_k	310
TECH	anna_k	245
TECH	tomek_dev	198
TECH	anna_k	178
TECH	pawel_x	134

```

=====
Problem P4 - Terminal Operations
=====

--- [1] count() - number of NEWS posts ---
Number of NEWS posts: 2

--- [2] anyMatch() - is there a post with tag 'docker'? ---
Is there a post with 'docker': true

--- [3] allMatch() - do all have at least 1 comment? ---
Do all have comments: true

--- [4] noneMatch() - do none of the posts have negative likes? ---
Do none have negative likes: true

--- [5] findFirst() - first LIFESTYLE post ---
Found: "Kraków dziś piękny :)"
Content (orElse): Kraków dziś piękny :)

--- [6] findFirst() - unverified user from Kraków ---
Unverified user from Kraków: bartek99

--- [7] Optional - method demonstration ---
isPresent(): false
orElse(): null
orElseGet(): None
orElseThrow() caught: Post does not exist!

```

35. 🚩 Milestone 4: **present the results of your work to the instructor**

36. Implement the code fragment marked with the *TODO 23* label, filtering the `posts` collection and storing the result in a new list.

- (a) Invoke the `stream()` method on the `posts` collection to obtain a stream of objects of type `Post`.
- (b) Apply the intermediate operation `filter(...)`.
 - Define the lambda expression `p -> p.getCategory().equals("TECH")`.
 - Restrict the stream to posts belonging to the "TECH" category.
- (c) Apply the terminal operation `collect(...)`.
 - Use `Collectors.toList()`.

- Collect the filtered elements into a new list.
- (d) Assign the result to the variable `techPosts` of type `List<Post>`.
- (e) Uncomment the line displaying information in the console.
37. Implement the code fragment marked with the *TODO 24* label, responsible for iterating through the `techPosts` list and printing its elements.
- (a) Invoke the `forEach(...)` method on the `techPosts` collection.
- (b) Pass a lambda expression as the method argument.
- (c) Inside the lambda expression, use `System.out.println(...)`.
- (d) Build the output text by concatenating an indentation of two spaces and the representation of the `p` object.
- Use the `toString()` method of the `p` object.
 - Ensure that each element is printed on a separate line.
38. Implement the code fragment marked with the *TODO 25* label, transforming the `posts` collection into a set of unique categories.
- (a) Invoke the `stream()` method on the `posts` collection to obtain a stream of objects of type `Post`.
- (b) Apply the intermediate operation `map(...)`.
- Use the method reference `Post::getCategory`.
 - Transform each `Post` object into its category of type `String`.
- (c) Apply the terminal operation `collect(...)`.
- Use `Collectors.toSet()`.
 - Collect the elements into a structure of type `Set`, eliminating duplicates.
- (d) Assign the result to the variable `categories` of type `Set<String>`.
- (e) Uncomment the line displaying information in the console.
39. Implement the code fragment marked with the *TODO 26* label, transforming the `posts` collection into a list of author usernames while preserving duplicates.
- (a) Invoke the `stream()` method on the `posts` collection.
- (b) Apply the intermediate operation `map(...)`.
- Define the lambda expression `p -> p.getAuthor().getUsername()`.
 - Transform each `Post` object into the author's username.
- (c) Apply the terminal operation `collect(...)`.
- Use `Collectors.toList()`.
 - Collect the results into a list of type `List<String>`.
 - Preserve all elements, including duplicate usernames.
- (d) Assign the result to the variable `authorsList`.

- (e) Uncomment the line displaying information in the console.
40. Implement the code fragment transforming the `posts` collection into a set of author usernames without duplicates.
- (a) Invoke the `stream()` method on the `posts` collection again.
 - (b) Apply the `map(...)` operation in the same way as before.
 - Retrieve the author's username through `p.getAuthor().getUsername()`.
 - (c) Apply the terminal operation `collect(...)`.
 - Use `Collectors.toSet()`.
 - Collect the results into a set of type `Set<String>`.
 - Ensure automatic removal of duplicates.
 - (d) Assign the result to the variable `authorsSet`.
 - (e) Uncomment [!!!]
41. Implement the code fragment marked with the *TODO 27* label, transforming the `posts` collection into a map storing the number of likes for each post identifier.
- (a) Invoke the `stream()` method on the `posts` collection to obtain a stream of objects of type `Post`.
 - (b) Apply the terminal operation `collect(...)`.
 - (c) Use `Collectors.toMap(...)` as the collector.
 - i. Specify the key-generating function as `Post::getId`.
 - Ensure that the post identifier of type `String` is used as the map key.
 - ii. Specify the value-generating function as `Post::getLikes`.
 - Ensure that the number of likes is stored as a value of type `Integer`.
 - (d) Assign the result to the variable `likesById` of type `Map<String, Integer>`.
42. Implement the code fragment printing the contents of the `likesById` map.
- (a) Invoke the `forEach(...)` method on the `likesById` map.
 - (b) Define a lambda expression taking the parameters (`id`, `likes`).
 - (c) Inside the lambda expression, use `System.out.printf(...)`.
 - (d) Use the format " %-5s -> %d %n".
 - Display the post identifier aligned to a width of 5 characters.
 - Display the corresponding number of likes.
43. Implement the code fragment marked with the *TODO 28* label, transforming the `posts` collection into a list of author usernames while preserving duplicates.
- (a) Invoke the `stream()` method on the `posts` collection.
 - (b) Apply the intermediate operation `map(...)`.

- Define the lambda expression `p -> p.getAuthor().getUsername()`.
 - Transform each `Post` object into the author's username.
- (c) Apply the terminal operation `collect(...)`.
- Use `Collectors.toList()`.
 - Collect the results into a list of type `List<String>`.
 - Preserve all elements, including duplicates.
- (d) Assign the result to the variable `authorsWithDuplicates`.
44. Implement the code fragment displaying information about the created list.
- (a) Invoke `authorsWithDuplicates.size()` in order to obtain the number of list elements.
- (b) Print a message containing the size of the list using `System.out.println`.
- (c) Print the full contents of the list using `System.out.println`.
- Allow the list's `toString()` method to be used to present all elements.
45. Implement the code fragment marked with the *TODO 29* label, creating an unmodifiable list of categories based on the `posts` collection.
- (a) Invoke the `stream()` method on the `posts` collection.
- (b) Apply the intermediate operation `map(...)`.
- Use the method reference `Post::getCategory`.
 - Transform each `Post` object into its category of type `String`.
- (c) Apply the intermediate operation `distinct()`.
- Remove duplicate categories from the stream.
- (d) Apply the terminal operation `collect(...)`.
- Use `Collectors.toUnmodifiableList()`.
 - Create an unmodifiable result list.
- (e) Assign the result to the variable `immutable` of type `List<String>`.
46. Implement the code fragment displaying the contents of the created list.
- (a) Invoke `System.out.println(...)`.
- (b) Concatenate the informational text with the contents of the `immutable` list.
47. Implement the code fragment demonstrating that the list cannot be modified.
- (a) Place the attempt to modify the list inside a `try` block.
- (b) Invoke the method `immutable.add("SPORT")`.
- (c) Take into account that the operation will cause an `UnsupportedOperationException` to be thrown.
- (d) Add a `catch(UnsupportedOperationException e)` block.

- (e) Inside the `catch` block, print a message containing the exception class name using `e.getClass().getSimpleName()`.

48. Check whether the program output will be:

```
=====
Problem P1 - Creating Streams
=====

--- [1] Stream.of() ---
anna_k
bartek99
zosia_m

--- [2] Arrays.stream() ---
JAVA
NEWS
PHOTO
TECH

--- [3] Stream.iterate() - post IDs ---
POST_1 POST_2 POST_3 POST_4 POST_5 POST_6 POST_7 POST_8 POST_9 POST_10

--- [4] Stream.generate() - random likes ---
15 59 4 89 11

--- [5] Stream.empty() + Optional ---
Does result exist: false
Value or default: No posts

--- [BONUS] Collection.stream() ---
anna_k
bartek99
zosia_m

=====
Problem P2 - filter(), map(), distinct()
=====

--- [1] Posts in TECH category (first 40 characters) ---
Java Stream API to świetna sprawa!
Mój ulubiony #java trick - metoda reduce
Docker + Spring Boot w 10 minut
Czy AI zastąpi programistów?
Wydajność streamów równoległych w Javie

--- [2] Unique post authors (distinct) ---
anna_k
bartek99
zosia_m
tomek_dev
kasia.foto
pawel_x

--- [3] Posts with more than 200 likes ---
```

```
anna_k      -> 245
zosia_m     -> 523
anna_k      -> 310
kasia.foto  -> 892
zosia_m     -> 445
bartek99    -> 201
```

```
--- [4] Post contents in UPPERCASE ---
JAVA STREAM API TO ŚWIETNA SPRAWA!
KRAKÓW DZIŚ PIĘKNY :)
NOWE PRZEPISY PODATKOWE - CO MUSISZ WIEDZIEĆ
MÓJ ULUBIONY #JAVA TRICK - METODA REDUCE()
DOCKER + SPRING BOOT W 10 MINUT
ZŁOTA GODZINA - MOJE ZDJĘCIA Z WROCŁAWIA
CZY AI ZASTĄPI PROGRAMISTÓW?
BREAKING NEWS - WYBORY SAMORZĄDOWE
TOP 5 KAWIARNI W KRAKOWIE
WYDAJNOŚĆ STREAMÓW RÓWNOLEGŁYCH W JAVIE
```

```
=====
Problem P3 - sorted(), limit(), skip()
=====
```

```
--- [1] TOP 3 posts by likes ---
```

```
kasia.foto  -> 892
zosia_m     -> 523
zosia_m     -> 445
```

```
--- [2] Posts from newest ---
```

```
2024-03-17 | Wydajność streamów równoległych w J
2024-03-16 | Top 5 kawiarni w Krakowie
2024-03-15 | Czy AI zastąpi programistów?
2024-03-15 | Breaking news - wybory samorządowe
2024-03-14 | Docker + Spring Boot w 10 minut
2024-03-14 | Złota godzina - moje zdjęcia z Wroc
2024-03-13 | Mój ulubiony #java trick - metoda r
2024-03-12 | Nowe przepisy podatkowe - co musisz
2024-03-11 | Kraków dziś piękny :)
2024-03-10 | Java Stream API to świetna sprawa!
```

```
--- [3] Posts starting from the 4th (skip 3) ---
```

```
[p4] Mój ulubiony #java trick - metoda r
[p5] Docker + Spring Boot w 10 minut
[p6] Złota godzina - moje zdjęcia z Wroc
[p7] Czy AI zastąpi programistów?
[p8] Breaking news - wybory samorządowe
[p9] Top 5 kawiarni w Krakowie
[p10] Wydajność streamów równoległych w J
```

```
--- [4] Pagination - page 2, size 4 ---
```

```
[p5] Docker + Spring Boot w 10 minut (198)
[p6] Złota godzina - moje zdjęcia z Wroc (892)
[p7] Czy AI zastąpi programistów? (134)
```

[p8] Breaking news - wybory samorządowe (445)

--- [BONUS] Sorting: category -> likes descending ---

LIFESTYLE	kasia.foto	892
LIFESTYLE	bartek99	201
LIFESTYLE	bartek99	87
NEWS	zosia_m	523
NEWS	zosia_m	445
TECH	anna_k	310
TECH	anna_k	245
TECH	tomek_dev	198
TECH	anna_k	178
TECH	pawel_x	134

=====

Problem P4 - Terminal Operations

=====

--- [1] count() - number of NEWS posts ---

Number of NEWS posts: 2

--- [2] anyMatch() - is there a post with tag 'docker'? ---

Is there a post with 'docker': true

--- [3] allMatch() - do all have at least 1 comment? ---

Do all have comments: true

--- [4] noneMatch() - do none of the posts have negative likes? ---

Do none have negative likes: true

--- [5] findFirst() - first LIFESTYLE post ---

Found: "Kraków dziś piękny :)"

Content (orElse): Kraków dziś piękny :)

--- [6] findFirst() - unverified user from Kraków ---

Unverified user from Kraków: bartek99

--- [7] Optional - method demonstration ---

isPresent(): false

orElse(): null

orElseGet(): None

orElseThrow() caught: Post does not exist!

=====

Problem P5 - collect() basics

=====

Number of TECH posts: 5

Post{id='p1', author='anna_k', likes=245, comments=30, category='TECH', date=2024-03-10}

Post{id='p4', author='anna_k', likes=310, comments=45, category='TECH', date=2024-03-13}

Post{id='p5', author='tomek_dev', likes=198, comments=22, category='TECH', date=2024-03-14}

Post{id='p7', author='pawel_x', likes=134, comments=88, category='TECH', date=2024-03-15}

Post{id='p10', author='anna_k', likes=178, comments=41, category='TECH', date=2024-03-17}

--- [2] toSet() - unique categories ---

Unique categories: [TECH, LIFESTYLE, NEWS]

--- [2b] Comparison List vs Set with duplicates ---

List (with duplicates): 10 elements -> [anna_k, bartek99, zosia_m, anna_k, tomek_dev, kasia.foto, pawel_x, zosia_m, bartek99, anna_k]

Set (without duplicates): 6 elements -> [zosia_m, kasia.foto, pawel_x, anna_k, tomek_dev, bartek99]

--- [3] toMap() - post id -> number of likes ---

p1 -> 245

p2 -> 87

p3 -> 523

p4 -> 310

p5 -> 198

p6 -> 892

p7 -> 134

p8 -> 445

p9 -> 201

p10 -> 178

--- [4] List of authors (with duplicates) ---

Size of authors list (with duplicates): 10

List: [anna_k, bartek99, zosia_m, anna_k, tomek_dev, kasia.foto, pawel_x, zosia_m, bartek99, anna_k]

--- [BONUS] toUnmodifiableList() ---

Unmodifiable list of categories: [TECH, LIFESTYLE, NEWS]

Attempt to modify unmodifiable list: UnsupportedOperationException

49. ★ Milestone 5: present the results of your work to the instructor